

The Rank-22 3×3 Matrix-Multiplication Problem over $\mathbb{F}_2$: Logical Form

Greg Sidebottom

Research note — logic repo (<https://github.com/gsidebottom/logic>), matmul track, 2026-08-22. The problem behind the `matmul/r22/` instances and the certified-tree program (`proof/Mm/Rank22Covering.lean`), stated above the CNF level: the tensor equation, the Brent equations, the symmetry group, and the stratum and wound variants the `hydra_satsuma` probes actually ran on.

1 The statement being decided

Fix $r = 22$. The decision problem is the existential sentence

$$\exists \alpha^{(1)}, \dots, \alpha^{(r)}, \beta^{(1)}, \dots, \beta^{(r)}, \gamma^{(1)}, \dots, \gamma^{(r)} \in \mathbb{F}_2^{3 \times 3} \quad \text{such that} \quad \bigoplus_{m=1}^r \alpha^{(m)} \otimes \beta^{(m)} \otimes \gamma^{(m)} = \mathcal{T}_{\langle 3,3,3 \rangle}, \quad (1)$$

where $\oplus$ is entrywise addition in $\mathbb{F}_2$ (XOR) and $\mathcal{T}_{\langle 3,3,3 \rangle}$ is the *matrix-multiplication tensor*: a fixed $9 \times 9 \times 9$ bit array indexed by an entry position of A , one of B , and one of C ,

$$(\mathcal{T}_{\langle 3,3,3 \rangle})_{(a,b),(c,d),(p,q)} = [b = c] \wedge [a = p] \wedge [d = q], \quad a, b, c, d, p, q \in \{1, 2, 3\}. \quad (2)$$

$\mathcal{T}_{\langle 3,3,3 \rangle}$ has a 1 exactly where the monomial $A_{ab}B_{cd}$ occurs in the product entry C_{pq} — the 27 naive terms $A_{pk}B_{kq}$ — and 0 at the other 702 positions.

Meaning of the unknowns. Each triple $(\alpha^{(m)}, \beta^{(m)}, \gamma^{(m)})$ describes one multiplication. $\alpha^{(m)}$ selects which entries of A are summed into its left factor, $\beta^{(m)}$ which entries of B are summed into its right factor, and $\gamma^{(m)}$ which output entries receive the product:

$$M_m = \left(\sum_{a,b} \alpha_{ab}^{(m)} A_{ab} \right) \left(\sum_{c,d} \beta_{cd}^{(m)} B_{cd} \right), \quad C_{pq} = \sum_{m=1}^r \gamma_{pq}^{(m)} M_m. \quad (3)$$

Equation (1) says that (3) computes $C = AB$ identically — for symbolic, non-commuting A and B — using r multiplications. “Rank 22” is literally the number of rank-one tensors in the sum. A summand with $\alpha^{(m)} = 0$ contributes nothing, so the sentence asks for rank ≤ 22 . The unknowns are $3 \times 9 \times 22 = 594$ bits.

2 The same sentence as a system of equations (Brent)

Reading (1) coordinate by coordinate gives one equation per index tuple $(a, b, c, d, p, q) \in \{1, 2, 3\}^6$ — **729 equations**, each an XOR of r three-way conjunctions equated to a constant:

$$\bigoplus_{m=1}^{22} \left(\alpha_{ab}^{(m)} \wedge \beta_{cd}^{(m)} \wedge \gamma_{pq}^{(m)} \right) = [b = c] \wedge [a = p] \wedge [d = q] \quad (4)$$

These are the *Brent equations* over $\mathbb{F}_2$. They split by their right-hand side:

- **27 target equations** (RHS = 1; the tuples with $b = c, a = p, d = q$): each naive term $A_{pk}B_{kq}$ must be generated by an *odd* number of the 22 multiplications.
- **702 cancellation equations** (RHS = 0): every other cross term $A_{ab}B_{cd}$ must be generated an *even* number of times, so that it cancels in every output.

Algebraically, (4) is a system of 729 **multilinear cubic polynomials over** $\mathbb{F}_2$ in 594 unknowns, written in algebraic normal form (XOR of AND-monomials). The structure is rigid: every monomial takes exactly one variable from each of the three factors of a *single* product index m , and every equation contains exactly one monomial per product. Logically it is a pure $\exists$ sentence over 594 propositional variables; the only parameters are $r = 22$ and the constant tensor $\mathcal{T}_{(3,3,3)}$.

What a verdict would mean.

- **SAT:** a rank-22 scheme over $\mathbb{F}_2$ — a genuine discovery. Heule–Kauers–Seidl lift such $\mathbb{F}_2$ solutions to $\mathbb{Z}$ by a Gröbner-basis step that rarely fails, so it would very likely give an integer scheme beating Laderman’s 23.
- **UNSAT:** no scheme over $\mathbb{F}_2$, hence none with integer coefficients (an integer scheme reduces mod 2 to an $\mathbb{F}_2$ scheme of rank ≤ 22). It would *not* rule out schemes over fields of odd characteristic, or rational schemes with even denominators.

Today $r = 22$ is open in both directions: Laderman’s 23 is the best upper bound, Bläser’s 19 the best lower bound.

3 The symmetry group

The solution set of (4) is invariant under a large group, which is what licenses the “without loss of generality” pins of the stratum instances.

1. **Product relabeling** S_{22} : the sum in (1) does not care about the order of its summands.
2. **Change of basis.** For $U, V, W \in GL_3(\mathbb{F}_2)$ (168 elements each), substituting $A \mapsto UAV$, $B \mapsto V^{-1}BW$ turns a scheme for AB into a scheme for $U(AB)W$; undoing that on the output gives a new scheme for AB . On the coefficient triples the substitution acts by *sandwiching*,

$$(\alpha, \beta, \gamma) \mapsto (U^T \alpha V^T, V^{-T} \beta W^T, U^{-1} \gamma W^{-1}), \quad (5)$$

applied to every summand m . The probes use the $W = I$ slice — the *A-side sandwich* $\alpha \mapsto U^T \alpha V^T$, $\beta \mapsto V^{-T} \beta$, $\gamma \mapsto U^{-1} \gamma$ of **strata.py** and **Rank22Covering.lean** — which already moves α by an arbitrary pair of invertible matrices on its two sides.

3. **Tensor symmetries** S_3 : $\mathcal{T}_{(3,3,3)}$ is invariant under cycling the roles of A, B, C (because $\text{tr}(ABC)$ is cyclic, with a transpose on the output slot) and under transposition $(A, B, C) \mapsto (B^T, A^T, C^T)$.

The rank of $\alpha^{(m)}$ is unchanged by (5) (it multiplies α by invertible matrices), so the multiset of α -ranks is an invariant of a solution’s orbit. That is the handle for the root split.

4 The root strata (cube-and-conquer, level 1)

Split on $\rho = \min_m \text{rank } \alpha^{(m)} \in \{0, 1, 2, 3\}$ and canonicalize the minimizing summand into slot 1 with (5). Each stratum is (4) plus a small conjunction:

stratum	added constraint	reading
S_0	$\alpha^{(1)} = 0$	a dead product — the rank-21 instance
S_1	$\alpha^{(1)} = E_{11}$	a rank-1 factor, canonicalized to the corner matrix
S_2	$\bigwedge \text{rank } \alpha^{(m)} \geq 2 \wedge \alpha^{(1)} = \text{diag}(1, 1, 0)$	no rank-0/1 factor anywhere; witness of rank 2 in slot 1
S_3	$\bigwedge_m \det \alpha^{(m)} = 1 \wedge \alpha^{(1)} = I_3$	every factor invertible; witness in slot 1

The rank predicates are themselves small Boolean formulas over the nine bits of a 3×3 matrix:

$$\text{rank } \alpha \geq 2 \equiv \bigvee_{\substack{i < k \\ j < l}} (\alpha_{ij} \alpha_{kl} \oplus \alpha_{il} \alpha_{kj}) \quad (\text{some } 2 \times 2 \text{ minor is nonzero; 9 minors}), \quad (6)$$

$$\det \alpha = 1 \equiv \bigoplus_{\sigma \in S_3} \alpha_{1\sigma(1)} \alpha_{2\sigma(2)} \alpha_{3\sigma(3)} \quad (\text{six permutation terms; signs vanish over } \mathbb{F}_2). \quad (7)$$

Covering. Every solution of (4) is carried by some element of $S_{22} \times GL_3(\mathbb{F}_2)^2$ into one of $S_0, \dots, S_3$ (this is the **covering** theorem of **Rank22Covering.lean**; **brent_perm** is proved, the sandwich and canonical-form lemmas are the formalization queue). Hence four certified UNSAT leaves would prove (1) false. Measured so far: each stratum ran **hydra_satsuma** (satsuma symmetry breaking + kissat, SR-certified) for 2 h without a verdict, as did the unstratified $r = 22$ instance at 4 h and the satisfiable $r = 23$ control at 6 h — level 1 of the tree is not decidable at hours scale.

5 Wound instances (the calibration family)

Take a known rank-23 scheme $(\hat{\alpha}, \hat{\beta}, \hat{\gamma})$, delete one of its products, and freeze n_{fix} of the remaining 22 summands at their known values:

$$\exists (\alpha^{(m)}, \beta^{(m)}, \gamma^{(m)})_{m \notin P} : \bigoplus_{m \in P} \hat{\alpha}^{(m)} \otimes \hat{\beta}^{(m)} \otimes \hat{\gamma}^{(m)} \oplus \bigoplus_{m \notin P} \alpha^{(m)} \otimes \beta^{(m)} \otimes \gamma^{(m)} = \mathcal{T}_{(3,3,3)}, \quad |P| = n_{\text{fix}}. \quad (8)$$

Only $22 - n_{\text{fix}}$ summands are free, so the instance asks whether a local repair of the wound exists. (“Wound” as in injury: the metaphor is to take a healthy rank-23 scheme, wound it by deleting one of its 23 products, freeze most of the survivors, and ask the solver whether the few free products can heal the damage — complete the tensor with 22. The term is this project’s coinage, not standard terminology; the closest published relative is the deletion/repair or completion notion in the flip-graph literature.) This family is how the solver was priced: 414 attacks, 273 certified UNSAT and 141 timeouts, with difficulty rising roughly $15\times$ for every 4 additional free products and the 90-second frontier sitting at $n_{\text{fix}} \approx 14\text{--}16$.

6 Down to CNF, in one paragraph

The only step below this level is Tseitin. Each pair-AND $\alpha_{ab}^{(m)} \wedge \beta_{cd}^{(m)}$ gets a definitional variable (81 per product, shared across the nine γ entries), each triple is that variable $\wedge \gamma_{pq}^{(m)}$, and each 22-term XOR in (4) is chained through 3-input XOR gadgets. That is how 594 variables become the 31,941-variable, 126,117-clause **brent_3x3x22.cnf**, and why conflict-driven search struggles: the parity structure is invisible to clause learning, and Gaussian elimination cannot reach through the AND guards. Heule–Kauers–Seidl’s diagnosis (average backtrack level above 100, runtime roughly exponential in it) holds here too: local search with streamliners finds rank-23 witnesses, but nothing so far produces an UNSAT certificate at rank 22 — which is exactly why the certified tree splits the problem by symmetry instead of asking a solver to swallow it whole.

A The tractable case: 2×2 in **lib/matmul.jq**

Added 2026-09-14. The same sentence for $\langle 2, 2, 2 \rangle$ — $n = 2$, so $2^6 = 64$ Brent equations over $12r$ unknowns, 8 of them target equations — is small enough to decide in seconds, and it is the benchmark family on which the *box* backend of the web app (the matrix-method solver with compiled table constraints, **doc/box_backend_design.md**) is compared with CaDiCaL. Rank 7 is Strassen’s scheme and 7 is the rank over $\mathbb{F}_2$, so rank 6 is UNSAT — and with it every smaller rank, a summand with $\alpha^{(m)} = 0$ being allowed.

Variables and equations. The library **lib/matmul.jq** (deps: **expr.jq**, **math.jq**) writes the unknowns as **a_m_k**, **b_m_k**, **c_m_k** for product $m = 1..r$ and entry $k = 0..n^2 - 1$, k the row-major index of the entry ($k = n(i-1) + (j-1)$). **brent_eqs(n)** enumerates the tuples (a, b, c, d, p, q) and yields one record per equation,

$\{e, k_a, k_b, k_c, \text{rhs}\}$: the equation number, the entry indices of the three factors (the same for every m), and $\text{rhs} = [b = c][a = p][d = q]$. Every generator below is a **prod** over these records; t_e_m names the term $\alpha_{k_a}^{(m)} \beta_{k_b}^{(m)} \gamma_{k_c}^{(m)}$ of equation e .

Box forms. A *box* is a compiled table constraint called by name in the formula language, `name(arg; ...)`; the backend propagates a box’s table to arc consistency, or — when both the table and its negation are tiny — as the equivalent clauses. Two encodings:

- `mm_boxes(n; r)`: each term is an `and3` box, `and3(a;b;c;t) := t = abc`, and each equation one parity box with its right-hand side as a constant argument, `xor{r}(t_e_1;...;t_e_r;rhs)`. The parity boxes `xor3...xor8` are declared as chains of `xor2(p;t;q) := q = p  $\oplus$  t` with the running parities hidden — `xor_chain(r)` generates `xor2(t_1;t_2;p_2) xor2(p_2;t_3;p_3) ...xor2(p_{r-1};t_r;d)` — and are compiled by composition (joining the `xor2` tables and projecting the hidden parities), 2^{r-1} rows over $r + 1$ columns. For $r = 8$: 512 calls, 532 variables.
- `mm_chain(n; r)`: each equation is a chain of `xstep(p;a;b;c;q) := q = p  $\oplus$  abc` boxes, the running parity `p_e_m` an ordinary variable, the first p the constant 0 and the last q the right-hand side — $64r$ boxes of 5 columns.

CNF forms (for CaDiCaL). CaDiCaL is given exact clauses rather than the Tseitin encoding of a parity DNF: `mm_cnf(n; r)` writes each term as the four clauses of $t = abc$ and each equation as an `xor2` chain ($p_2 = t_1 \oplus t_2$, $p_m = p_{m-1} \oplus t_m$, $p_r = \text{rhs}$, four clauses per step, two where the constant absorbs q); `mm_cnf_direct(n; r)` writes the parity as one clause per wrong assignment of the r term variables (2^{r-1} clauses of r literals).

Symmetry breaking. `mm_sym_boxes(r)` is $\bigwedge_{m < r} \text{le}_4(a_m; a_{m+1})$ — the products in non-decreasing order of their α vectors read as 4-bit numbers (`le_4` is the $\leq$ box of `math.jq`); `mm_sym(r)` is the same as a formula for CaDiCaL. It is the product-relabelling symmetry S_r of §3: the sum does not depend on the order of its terms, so every scheme has a reordering that obeys it and satisfiability is preserved. `mm_boxes_sym`, `mm_chain_sym` and `mm_cnf_sym` attach it. The examples bar of the web app has “2x2 rank 8 boxes”, “2x2 rank 7 boxes” and “2x2 rank 6 boxes” (the last with the symmetry breaking); any of the generators can be typed into the jq filter box, e.g. `mm_boxes_sym(2; 6)`.

```
def mm_boxes(n; r):
  prod(brent_eqs(n) [] | . as $q |
    (range(r) + 1 | "and3(\mm_a($q; .));\mm_b($q; .);\mm_c($q; .);\mm_t($q; .)")),
    "xor\(\([range(r) + 1 | mm_t($q; .)] | join(";");\($q.rhs))");

def mm_sym_boxes(r): prod(range(1; r) | "le_4(a_\(.);a_\(. + 1))");
```

The box declarations of the library’s `# === boxes ===` block:

- `and3(a;b;c;t) := "t = a b c"`
- `xstep(p;a;b;c;q) := "q = p  $\oplus$  a b c"`
- `xor2(p;t;q) := "q = p  $\oplus$  t"`
- `xor7(t_1;t_2;t_3;t_4;t_5;t_6;t_7;d) := xor_chain(7)`, and likewise `xor3...xor8`

How the box backend decides it. The formula is a conjunction of calls, so the collapsed matrix has one path and the work is the completion engine’s: a conflict-driven search in which table boxes are propagated to arc consistency with bit-parallel live-row masks, tiny boxes (`and3`: 4 rows, 4 rows of negation) as watched clauses, conflicts and forced literals explained lazily from the tables’ kill masks, with 1-UIP learning, backjumping, VSIDS, Luby restarts and LBD-based clause deletion. The parity tables give it exactly the propagation of an XOR constraint — which, on this family, is no more than unit propagation on the exact CNF has.

Results. Times as the web app reports them (the job’s wall-clock, including building the tables), 60-second cap, best of one run per cell; boxes with the learned-clause budget 4000. “sym” is the symmetry breaking attached.

r	verdict	boxes backend				CaDiCaL			
		mm_chain	+sym	mm_boxes	+sym	mm_cnf	+sym	mm_cnf_direct	+sym
8	SAT	0.06 s	0.12 s	0.41 s	6.5 s	0.07 s	0.01 s	0.02 s	0.07 s
7	SAT	4.3 s	4.4 s	19.3 s	30.7 s	0.13 s	0.39 s	0.04 s	0.06 s
6	UNSAT	> 60 s	1.8 s	> 60 s	3.0 s	52.8 s	0.26 s	55.9 s	0.37 s
5	UNSAT	2.4 s	0.11 s	4.1 s	0.14 s	0.59 s	0.04 s	0.35 s	0.07 s

(Rank-5 box times are from the run before the deletion budget was raised.) Rank 8 and 7 SAT and rank 6 UNSAT are all decided within a minute by both engines; the UNSAT proof needs the symmetry breaking on both — without it CaDiCaL takes 53–56 s and the box engine does not finish. CaDiCaL is 10–100× faster on this family, and SAT times are search luck (the four box variants of rank 7 span 4–31 s). The benchmark exposed two defects in the box backend, both fixed: cancellation did not reach the completion engine, and the learned database grew without bound (propagation fell to a seventh of its speed on long runs).

Reading a scheme off the witness. A satisfying assignment *is* a scheme: $\mathbf{a_m_k}$ are the entries of $\alpha^{(m)}$, likewise $\beta^{(m)}$, $\gamma^{(m)}$, so product m is $(\sum_k \alpha_k^{(m)} A_k)(\sum_k \beta_k^{(m)} B_k)$ and $C_{pq} = \bigoplus_m \gamma_{pq}^{(m)} M_m$. `tools/mm_scheme.py` runs a formula of the library against the web app (the boxes backend, or CaDiCaL with `--cadical`; `--url` for a port other than 3001), reads n and r off the witness’s variable names, prints the scheme and checks it twice — every Brent equation, and $C = AB$ on 200 random pairs over $\mathbb{F}_2$. In the printout $+$ is addition in $\mathbb{F}_2$ (XOR). The same formula serves both engines: for CaDiCaL the box calls are expanded to their definitions (`and3`, the `xor2` chains, `le_4`) and Tseitin-encoded, which is even smaller than the explicit CNF forms (30K characters against 93K for rank 8) and solves as fast; the results table used `mm_cnf` for its CaDiCaL columns only so that those numbers do not depend on our expansion.

```
$ tools/mm_scheme.py 'mm_boxes_sym(2; 7)'
boxes backend: SAT in 30.56s
2x2 matrices, 7 products
M1 = (A11) (B11 + B21)
M2 = (A11 + A12) (B21)
M3 = (A21) (B11 + B12)
M4 = (A22) (B21 + B22)
M5 = (A12 + A22) (B12 + B22)
M6 = (A11 + A12 + A22) (B12 + B21)
M7 = (A11 + A12 + A21 + A22) (B12)
C11 = M1 + M2
C12 = M2 + M4 + M5 + M6
C21 = M2 + M3 + M6 + M7
C22 = M2 + M4 + M6 + M7
Brent equations: all satisfied
C = AB on 200 random pairs over GF(2): ok; 7 non-zero products
```

```
$ tools/mm_scheme.py 'mm_boxes_sym(2; 7)' --cadical
CaDiCaL: SAT in 0.12s
2x2 matrices, 7 products
M1 = (A12) (B11 + B12 + B21 + B22)
M2 = (A11 + A12) (B11 + B12)
M3 = (A21) (B11)
M4 = (A11 + A21) (B12 + B22)
M5 = (A11 + A12 + A21) (B11 + B12 + B22)
M6 = (A22) (B21)
M7 = (A11 + A12 + A21 + A22) (B22)
C11 = M1 + M3 + M4 + M5
C12 = M2 + M3 + M4 + M5
C21 = M3 + M6
```

```

C22 = M2 + M3 + M5 + M7
Brent equations: all satisfied
C = AB on 200 random pairs over GF(2): ok; 7 non-zero products

$ tools/mm_scheme.py 'mm_boxes_sym(2; 6)'
boxes backend: UNSAT in 2.92s
$ tools/mm_scheme.py 'mm_boxes_sym(2; 6)' --cadical
CaDiCaL: UNSAT in 0.30s

```

(The two engines return different 7-product schemes; each is checked as above.)

Reading it against rank 22. Everything above scales by the formula: `brent_eqs(3)` is the 729 equations; `mm_boxes(3; 22)` the 16,038 and 3 boxes and, per equation, the 22-term parity as a chain of three `xor8` blocks carrying intermediate parities (a single `xor22` table would have 2^{21} rows; the blocks propagate exactly as it would); `mm_sym_boxes(3; 22)` the S_{22} ordering over 9-bit α vectors (`1e_9`, whose negative table comes from the declaration's `negation lt(b; a; 9)` clause). The rank-21 instance `mm_boxes_sym(3; 21)` — 17,516 calls — parses in 4 ms, builds its tables in 0.2 s and searches; it is the S_0 stratum of §4, which ran for two hours without a verdict, so no verdict is expected. What does not scale is the solving: with the 2×2 UNSAT proof already needing the symmetry breaking for both engines, the rank-22 instance is the stratum-and-cube programme of §4–§5, not a single solver call.